# Custom 32-bit Virtual Machine

Instruction Set Architecture (ISA) Reference Manual

Ton Martín Gonzàlez

Software & Architecture Design

July 22, 2026

## Contents

# 1 Architecture Overview

This document outlines the architecture and instruction set of a custom 32-bit Virtual Machine (VM). The architecture follows the Von Neumann model, utilizing a single memory space that shares both instructions and data. It operates primarily as a Stack Machine, which means that all operations are based on a Stack data structure. The Stack is used to evaluate expressions and manage execution flow through a LIFO (Last-In, First-Out) structure and also provides general-purpose registers for local variable storage.

# 2 Programmer's Model

The Programmer's Model defines the architectural state of the Virtual Machine as seen by the software developer. It establishes the abstract interface between the executing program and the Virtual Machine engine. Understanding this model is crucial for writing efficient assembly code, as it dictates how data is organized, accessed, and modified during execution. The architectural state consists of a unified memory space, a Last-In-First-Out (LIFO) stack for expression evaluation, and a dedicated set of general-purpose registers.

## 2.1 Memory Space

The Virtual Machine uses a linear, byte-addressable memory model where both executable instructions and data reside side-by-side.

Multi-byte values, such as 32-bit integers (4 bytes) and memory addresses, are stored using **Little-Endian** byte ordering. This means that the least significant byte (LSB) of a multi-byte value is stored at the lowest memory address, while the most significant byte (MSB) is stored at the highest address.

*Example:* If the 32-bit hexadecimal value `0x12345678` is stored starting at memory address `0x0000`, it will be laid out in memory as follows:

- Address `0x0000`: `0x78` (LSB)
- Address `0x0001`: `0x56`
- Address `0x0002`: `0x34`
- Address `0x0003`: `0x12` (MSB)

### 2.2 The Stack

The Stack is the primary data structure for arithmetic and logic operations in this architecture. It is designed to store 32-bit signed integers. Unlike traditional register-based architectures, most ALU operations in this Virtual Machine implicitly target the top values of the stack. For example when it adds two numbers or it checks a condition for a jump.

- **SP (Stack Pointer):** An internal pointer that tracks the top of the stack. It automatically increments when a value is pushed and decrements when a value is popped. The developer does not manipulate the SP directly, but rather through stack-specific instructions.

### 2.3 Registers

To complement the stack and provide fast, temporary data storage without continuously pushing and popping memory, the architecture features a set of general-purpose registers.

- **R0 - RN:** General-purpose 32-bit registers. They are accessed via their integer index and are ideal for storing loop counters, pointers, and frequently accessed local variables.
- **IP (Instruction Pointer):** A 32-bit register holding the memory address of the next instruction to be fetched and executed. Modifying the IP directly via control flow instructions (`JMP`, `BEQ`, etc.) dictates the branching logic of the program.

## 3 Instruction Formats

Instructions consist of a 1-byte Opcode followed by optional operands. The VM recognizes the following formats:

- **1-Byte Format:** [Opcode]
- **2-Byte Format:** [Opcode] [Register Index (1B)]
- **5-Byte Format:** [Opcode] [32-bit Immediate/Address (4B)]
- **6-Byte Format:** [Opcode] [Register Index (1B)] [32-bit Immediate (4B)]

# 4 Instruction Set Reference

## 4.1 Stack Operations

### PUSH — Push Immediate to Stack

**Opcode:** `0x01`

**Size:** 5 Bytes

**Arguments:** `Immediate` (32-bit signed integer)

**Stack Effect:** `Stack` ← `Immediate`

**Description:**

Reads a 32-bit immediate value from the instruction stream and pushes it onto the top of the stack. The Stack Pointer (SP) is incremented.

### POP — Pop Value from Stack

**Opcode:** `0x02`

**Size:** 1 Byte

**Arguments:** None

**Stack Effect:** `Stack` →

**Description:**

Removes the 32-bit value at the top of the stack and discards it. The Stack Pointer (SP) is decremented. Throws a Stack Underflow error if the stack is empty.

### 4.2 Arithmetic Operations

## ADD — Integer Addition

**Opcode:** `0x03`

**Size:** 1 Byte

**Arguments:** None

**Stack Effect:** `[A], [B]` → `[A + B]`

**Description:**

Pops the top two values from the stack (`B` first, then `A`), adds them together, and pushes the 32-bit result back onto the stack.

## SUB — Integer Substraction

**Opcode:** `0x04`

**Size:** 1 Byte

**Arguments:** None

**Stack Effect:** `[A], [B]` → `[A - B]`

**Description:**

Pops the top two values from the stack (`B` first, then `A`), substracts `B` from `A`, and pushes the 32-bit result back onto the stack.

### 4.3 Data Transfer Operations

#### SETI — Set Value of Register with Immediate

**Opcode:** `0x05`

**Size:** 6 Bytes

**Arguments:** `Register Index` (1B), `Immediate` (32-bit signed integer)

**Stack Effect:** `None`

**Description:**

Reads a 32-bit `Immediate` value from the instruction stream and sets it into the Register with same `Register Index` as the argument.

#### PUSHR — Push value from Register to the Stack

**Opcode:** `0x07`

**Size:** 2 Bytes

**Arguments:** `Register Index` (1B)

**Stack Effect:** `Stack` ← `Register`

**Description:**

Reads an 8-bit index from the instruction stream and pushes the value from that `Register` on top of the Stack. The Stack Pointer (SP) is incremented.

#### POPR — Pop value from the Stack to a Register

**Opcode:** `0x08`

**Size:** 2 Bytes

**Arguments:** `Register Index` (1B)

**Stack Effect:** `Stack` → `Register`

**Description:**

Reads an 8-bit index from the instruction stream and pops the top of the Stack to that `Register`. The Stack Pointer (SP) is decremented.

## LOAD — Load value from Memory to the Stack

**Opcode:** `0x0B`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `Stack ← Memory`

**Description:**

Reads a 32-bit address from the instruction stream and pushes the value from that memory position to the Stack. The Stack Pointer (SP) is incremented.

## STORE — Store value from the Stack to the Memory

**Opcode:** `0x0C`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `Stack → Memory`

**Description:**

Reads a 32-bit address from the instruction stream and pops the top of the Stack to the memory position with that `Address`. The Stack Pointer (SP) is decremented.

### 4.4 Control Flow

#### HLT — Halt Execution

**Opcode:** `0x00`

**Size:** 1 Byte

**Arguments:** `None`

**Stack Effect:** `None`

**Description:**

Halts the execution of the Virtual Machine. The internal running state is set to false, stopping the instruction fetch-decode-execute cycle, and the virtual machine gracefully powers down.

#### JMP — Jump to another Instruction

**Opcode:** `0x06`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `None`

**Description:**

Jumps to another position of the memory indicated by the `Address` in the argument. To do so, it changes the value of the Instriction Pointer. The address can be set as a 32-bit integer value or as a Label in the program.

#### BZ — Branch if Equal to Zero

**Opcode:** `0x09`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `Stack` $\rightarrow$

**Description:**

Pops the top of the stack. If it is equal to 0, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BNZ — Branch if Not Equal to Zero

**Opcode:** `0x0A`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `Remove Top Value`

**Description:**

Pops the top of the stack. If it is not equal to 0, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BEQ — Branch if Equal

**Opcode:** `0x0D`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is equal to `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BNE — Branch if Not Equal

**Opcode:** `0x0E`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is not equal to `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BGT — Branch if Greater than

**Opcode:** `0x0F`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is greater than `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BLT — Branch if Less than

**Opcode:** `0x10`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is less than `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## BGE — Branch if Greater or Equal to

**Opcode:** `0x11`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is greater or equal to `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

### BLE — Branch if Less or Equal to

**Opcode:** `0x12`

**Size:** 5 Bytes

**Arguments:** `Address` (32-bit unsigned integer)

**Stack Effect:** `[A], [B]` →

**Description:**

Pops the top two values from the stack (`B` first, then `A`). If `A` is less or equal to `B`, the Instruction Pointer (IP) is set to `Address`. Otherwise, execution continues sequentially.

## 5 Appendix: Example Assembly Programs

Below is an example of an assembly program, demonstrating a basic conditional loop.

```
; Initialize Loop Counter in Register 0
SETI 0 5

loop_start:
    ; Get counter value and push to stack
    PUSHR 0
    PSH 0

    ; Branch to end if counter == 0 (BEQ)
    BEQ end_program

    ; Decrement counter: (Counter - 1)
    PUSHR 0
    PSH 1
    SUB
    POPR 0

    ; Jump back to the start of the loop
    JMP loop_start

end_program:
    HLT
```